Διερευνώντας τον Καιρό στην Αθήνα

Στην εργασία αυτή θα διερευνήσετε τον καιρό στην Αθήνα σε μία περίοδο που καλύπτει πάνω από 50 χρόνια.

Πάνος Λουρίδας, Αναπληρωτής Καθηγητής
Τμήμα Διοικητικής Επιστήμης και Τεχνολογίας
Οικονομικό Πανεπιστήμιο Αθηνών
louridas@aueb.gr

Ερωτήματα

E1: Συγκέντρωση των Δεδομένων

Θα εργαστείτε με δεδομένα που καλύπτουν την περίοδο από το 1955 έως το 2020. Τα δεδομένα προέρχονται από δύο πηγές:

- Από το National Oceanic and Atmospheric Administration's (NOAA) National Centers for Environmental Information (https://www.ncdc.noaa.gov/cdo-web/ (https://www.ncdc.noaa.gov/cdo-web/)) και πιο συγκεκριμένα https://www.ncdc.noaa.gov/cdo-web/search (https://www.ncdc.noaa.gov/cdo-web/search).
- Αφού μας ενδιαφέρει η Αθήνα, θα χρησιμοποιήσετε τα δεδομένα από τον μετεωρολογικό σταθμό του Ελληνικού και θα επικεντρωθείτε στη μέση ημερήσια θερμοκρασία. Τα δεδομένα για την περίοδο 1955 έως 2020 βρίσκονται στο αρχείο `noaa_hellinikon_1995_2020.csv.zip` (αλλά μπορείτε με την ευκαιρία να δείτε τι είδους δεδομένα προσφέρει δωρεάν το NOAA, είναι παράδειγμα προς μίμηση).
- Εξετάστε την πληρότητα των δεδομένων. Τι δεδομένα λείπουν;
- Για να συμπληρώσετε όσα δεδομένα από αυτά που λείπουν γίνεται, θα χρησιμοποιήσετε ένα δεύτερο σύνολο δεδομένων διαθέσιμο στο https://data.hellenicdataservice.gr/dataset/66e1c19a-7b0e-456f-b465-b301a1130e3f (https://data.hellenicdataservice.gr/dataset/66e1c19a-7b0e-456f-b465-b301a1130e3f). Αυτό το σύνολο καλύπτει μόνο την περίοδο από το 2010-2019. Θα πάρετε τα δεδομένα για την Αθήνα (`athens.csv` , θα πρέπει να το κατεβάσετε εσείς). Προσοχή στην τεκμηρίωση των δεδομένων.

In [1]:
```
import pandas as pd
import numpy as np
```

In [2]:
```
fts = pd.read_csv("./noaa_hellinikon_1995_2020.csv")
```

In [3]:
```
fts.shape
```

Out[3]:
```
(23251, 4)
```

Το dataframe αποτελείται από 23251 γραμμές και 4 στήλες.

In [4]:
```
fts.head()
```

Out[4]:

	STATION	NAME	DATE	TAVG
0	GR000016716	HELLINIKON, GR	1955-01-01	NaN
1	GR000016716	HELLINIKON, GR	1955-01-02	NaN
2	GR000016716	HELLINIKON, GR	1955-01-03	NaN
3	GR000016716	HELLINIKON, GR	1955-01-04	NaN
4	GR000016716	HELLINIKON, GR	1955-01-05	NaN

In [5]:
```
fts.isna().sum()
```

Out[5]:
```
STATION       0
NAME          0
DATE          0
TAVG       2311
dtype: int64
```

Υπάρχουν 2311 γραμμές με κενή την θερμοκρασία. Παραδείγματα γραμμών - ημερομηνιών που έχουν και δεν έχουν θερμοκρασία:

In [6]:
```
fts.loc[fts["DATE"] == "1991-02-20"]
```

Out[6]:

	STATION	NAME	DATE	TAVG
13199	GR000016716	HELLINIKON, GR	1991-02-20	11.2

In [7]:
```
fts.loc[fts["DATE"] == "2013-01-01"]
```

Out[7]:

STATION	NAME	DATE	TAVG

Εξετάζω το διάστημα 2010 - 2019, για το οποίο υπάρχουν επιπρόσθετα δεδομένα από το δεύτερο dataset:

In [8]:
```
from datetime import date, timedelta

start_date = date(2010, 1, 1)
end_date = date(2019, 1, 1)

delta = end_date - start_date ## returns timedelta object

missing_dates = []
null_temps = []
for i in range(delta.days + 1):
    day = start_date + timedelta(days = i)
    day_string = day.isoformat()
    if(len(fts.loc[fts["DATE"] == day_string]) == 0): missing_dates.append(day_string)
    elif(pd.isnull(fts.loc[fts["DATE"] == day_string]["TAVG"]).bool()): null_temps.append(day_string)
```

In [9]:
```
len(null_temps)
```

Out[9]:
```
0
```

In [10]:
```
len(missing_dates)
```

Out[10]:
```
853
```

Από τις παρούσες ημερομηνίες, δεν υπάρχει καμμία με κενή θερμοκρασία, αλλά απουσιάζουν εντελώς 853 ημέρες - σειρές (συμπεριλαμβανομένου σχεδόν ολόκληρου του έτους 2013). Θα συμπληρωθούν από το *athens.csv*, το οποίω κατεβάζω και αποθηκεύω στον φάκελο *assignment1*.

https://data.hellenicdataservice.gr/dataset/66e1c19a-7b0e-456f-b465-b301a1130e3f/resource/a7c024b3-8606-4f08-93e2-2042f5bd6748 (https://data.hellenicdataservice.gr/dataset/66e1c19a-7b0e-456f-b465-b301a1130e3f/resource/a7c024b3-8606-4f08-93e2-2042f5bd6748)

«Κάθε γραμμή του αρχείου αποτελείται από 14 στήλες. Η πρώτη στήλη είναι η ημέρα την οποία αφορούν οι παράμετροι (YYYY-MM-DD). Έπειτα έχουμε μέση, μέγιστη και ελάχιστη θερμοκρασία ημέρας (°C). Συνεχίζουν αντίστοιχες τιμές για την σχετική υγρασία και την πίεση (%, hPa). Μετά έχουμε την αθροιστική βροχόπτωση της ημέρας (mm). Οι τελευταίες τρεις στήλες αφορούν τον άνεμο. Δίνεται η μέση ταχύτητα και την επικρατούσα διεύθυνση του ανέμου, ενώ στην τελευταία στήλη, η μέγιστη ριπή (km/h). [EN] Each line consists of 14 columns. First is the date of the parameters (YYYY-MM-DD). Next are the mean, maximum and minimum temperature (°C). After that and in the same order we have 6 columns for relative humidity and atmospheric pressure (%, hPa). The tenth column is the daily rainfall (mm). In the last 3 columns we can see mean wind speed, dominant wind direction as well as wind gust (km/h)».

Από την τεκμηρίωση βλέπω ότι χρήσιμες για την εργασία είναι μόνον οι δύο πρώτες στήλες. Δεν υπάρχει πληροφορία για τον ή τους μετεωρολογικούς σταθμούς από όπου προέρχονται οι μετρήσεις, οπότε στην συμπλήρωση αυτές οι στήλες θα μείνουν κενές. Το αρχείο δεν περιέχει επικεφαλίδα, άρα πρέπει τα ονόματα των στηλών να συμπληρωθούν. Εξετάζοντας το αρχείο επίσης βλέπω ότι δεν απουσιάζουν ημερομηνίες από αυτό, και οι θερμοκρασίες που απουσιάζουν αντικαθίστανται με την συμβολοσειρά «---».

In [11]:
```
athens = pd.read_csv("./athens.csv", usecols = [0, 1], names = ["DATE", "TAVG"], na_values = ["---"])
```

In [12]:
```
athens.head()
```

Out[12]:

	DATE	TAVG
0	2010-01-01	17.9
1	2010-01-02	15.6
2	2010-01-03	13.5
3	2010-01-04	9.5
4	2010-01-05	13.4

Συμπληρώνω το κυρίως dataframe:

In [13]:
```
for md in missing_dates:
    if(pd.isnull(athens.loc[athens["DATE"] == md]["TAVG"]).bool()): continue
    fts = fts.append({"STATION": None, "NAME": None, "DATE": md, "TAVG": float(athens.loc[athens["DATE"] == md]["TAVG"].values[0]
```

In [14]:
```
fts.shape
```

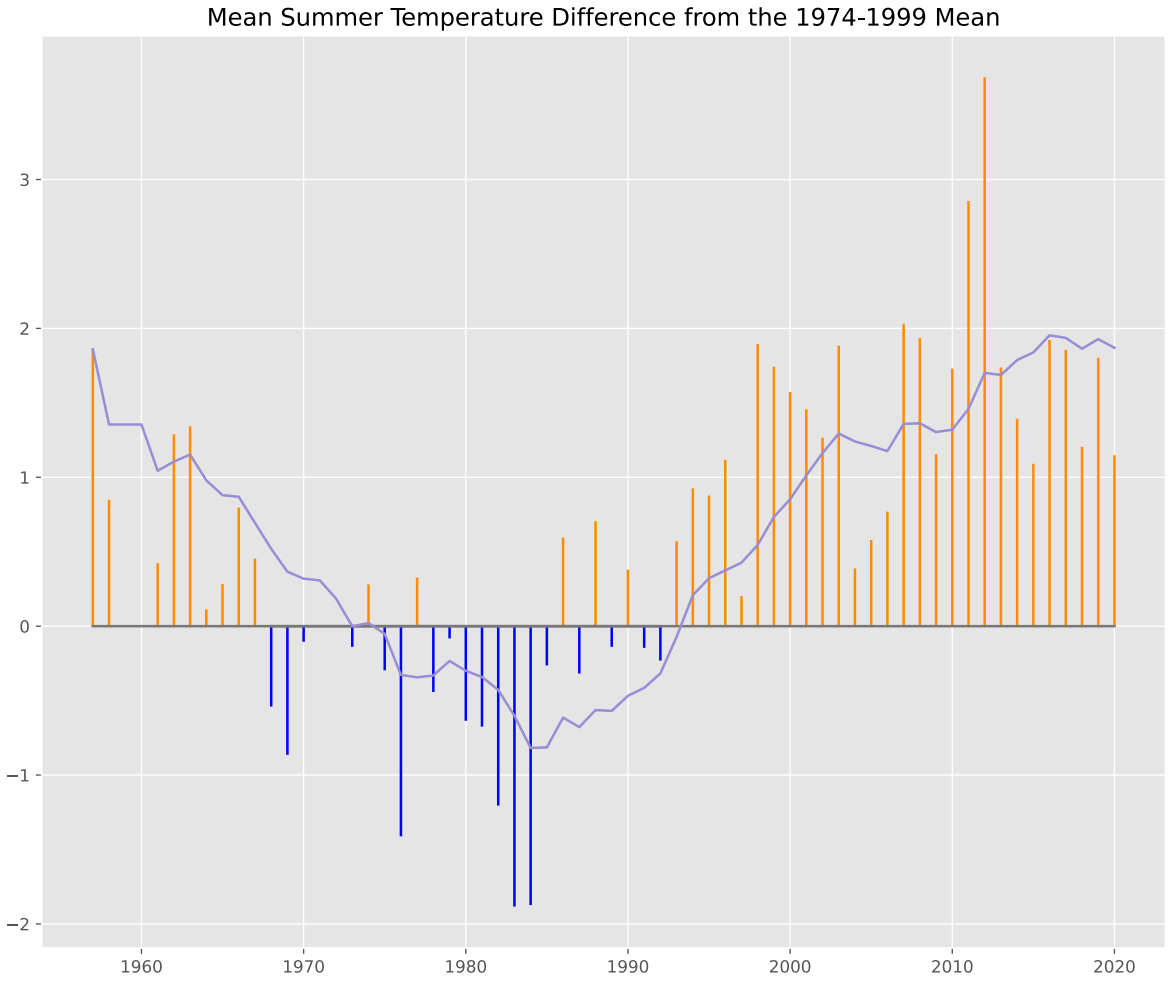
Out[14]:
```
(24103, 4)
```

Προστέθηκαν $24103 - 23251 = 852$ σειρές, ενώ απουσίαζαν 853, δηλαδή συμπληρώθηκαν σχεδόν όλες οι απουσιάζουσες ημερομηνίες στο διάστημα 2010 - 2019.

Ε2: Απόκλιση Καλοκαιρινών Θερμοκρασιών

Η Ελληνική Μετεωρολογική Υπηρεσία (ΕΜΥ) έχει εκδόσει μια αναφορά που πραγματεύεται τα ακραία καιρικά φαινόμενα για το 2020. Η αναφορά είναι διαθέσιμη στο http://www.hnms.gr/emy/en/pdf/2020_GRsignificantEVENT_en.pdf (http://www.hnms.gr/emy/en/pdf/2020_GRsignificantEVENT_en.pdf). Στη σελίδα 7 της αναφοράς υπάρχει ένα γράφημα το οποίο εμφανίζει την απόκλιση της μέσης ημερήσιας καλοκαιρινής θερμοκρασίας από τη θερμοκρασία αναφοράς που προκύπτει από τα έτη 1971-2000.

Θα φτιάξετε τη δική σας έκδοση του γραφήματος, χρησιμοποιώντας ως θερμοκρασία αναφοράς τη μέση ημερήσια καλοκαιρινή θερμοκρασία των ετών 1974-1999. Το γράφημά σας θα πρέπει να είναι παρόμοιο με αυτό που φαίνεται παρακάτω. Η μπλε γραμμή στο γράφημα είναι ο κυλιομένος μέσος όρος δέκα ετών (10 years rolling average). Πώς ερμηνεύετε το γράφημα;



Όπως είναι προφανές, αλλά διευκρινίζεται και από την δεδομένη αναφορά, ο μέσος όρος καλοκαιρινής θερμοκρασίας υπολογίζεται από τους τρεις καλοκαιρινούς μήνες: 1η Ιουνίου έως και 31η Αυγούστου.

Υπολογίζω επαναληπτικώς μέση θερμοκρασία αναφοράς από 1974 έως 1999, και μέση θερμοκρασία κάθε έτους από 1955 έως και 2020.

In [15]:
```
missing_dates_2 = []
null_temps_2 = []
avgs_per_year = []
reference_temp_sum = 0
total_measurements = 0
reference_measurements = 0
null_years = []
for year in range(1955, 2021):
    year_temp_sum = 0
    year_measurements = 0
    start_date = date(year, 6, 1)
    end_date = date(year, 8, 31)

    delta = end_date - start_date ## returns timedelta object

    for i in range(delta.days + 1):
        day = start_date + timedelta(days = i)
        day_string = day.isoformat()
        if(len(fts.loc[fts["DATE"] == day_string]) == 0): missing_dates_2.append(day_string)
        elif(pd.isnull(fts.loc[fts["DATE"] == day_string]["TAVG"]).bool()): null_temps_2.append(day_string)
        else:
            year_temp_sum += fts.loc[fts["DATE"] == day_string]["TAVG"].values[0]
            total_measurements += 1
            year_measurements += 1
            if(1974 <= year and year <= 1999):
                reference_temp_sum += fts.loc[fts["DATE"] == day_string]["TAVG"].values[0]
                reference_measurements += 1

    if(year_measurements):
        avgs_per_year.append(year_temp_sum / year_measurements)
    else:
        null_years.append(year)
```

In [16]:
```
fts.loc[fts["DATE"] == "2013-06-01"]
```

Out[16]:

	STATION	NAME	DATE	TAVG
23601	None	None	2013-06-01	20.6

In [17]:
```
athens.loc[athens["DATE"] == "2013-06-01"]
```

Out[17]:

	DATE	TAVG
1247	2013-06-01	20.6

Από όλο το dataset απουσιάζει εντελώς μία μόνο ημέρα των τριών καλοκαιρινών μηνών.

In [18]:
```
len(missing_dates_2)
```

Out[18]: 1

In [19]:
```
missing_dates_2
```

Out[19]: ['2007-08-28']

Όμως υπάρχουν 590 ημέρες για τις οποίες δεν υπάρχει θερμοκρασία.

In [20]:
```
len(null_temps_2)
```

Out[20]: 590

Επαληθεύεται ότι έχει γίνει σωστά η καταμέτρηση: διάστημα 66 ετών επί 92 ημέρες ανά έτος (30 ο Ιούνιος + 31 ο Ιούλιος + 31 ο Αύγουστος), μείον τις 590 που λείπουν, μείον την μία που δεν υπάρχει.

In [21]:
```
total_measurements
```

Out[21]: 5481

In [22]:
```
66 * 92 - 590 - 1
```

Out[22]: 5481

Μέση θερμοκρασία αναφοράς στο διάστημα 1974 - 1999.

In [23]:
```
reference_measurements
```

Out[23]: 2386

In [24]:
```
reference_avg_temp = reference_temp_sum / reference_measurements
```

In [25]:
```
reference_avg_temp
```

Out[25]: 26.232145850796297

Διαφορά μέσης καλοκαιρινής θερμοκρασίας κάθε έτους από την αναφορά:

In [26]:
```
diffs = [i - reference_avg_temp for i in avgs_per_year]
```

In [27]:
```
len(diffs)
```

Out[27]: 60

Σχετικά με τα απόντα δεδομένα

Θα μπορούσα να χρησιμοποιήσω κάποια τεχνική interpolation ώστε να τα συμπληρώσω, όμως δεν θεωρώ ότι αυτό είναι σωστό. Ειδικά σε θέμα επιστημονικό, και δη περιβαλλοντολογικό. Συνεπώς τα έτη αυτά θα αφαιρεθούν εντελώς από τα δεδομένα (θα απουσιάζουν από τον οριζόντιο άξονα στα διαγράμματα).

In [28]:
```
null_years
```

Out[28]: [1955, 1956, 1959, 1960, 1971, 1972]

In [29]:
```
valid_years = sorted(list(set(range(1955, 2021)) - set(null_years)))
```

Κυλιόμενος αριθμητικός μέσος

In [30]:
```
rolling_avg = []
for i in range(len(diffs)):
    left = max(0, i - 9)
    rolling_avg.append(sum(diffs[left : i + 1]) / (i - left + 1))
```

In [31]:
```
len(rolling_avg)
```

Out[31]: 60

Διάγραμμα

In [32]:
```
import matplotlib
import matplotlib.pyplot as plt
import seaborn as sns
```

In [33]:
```
    matplotlib.rc_file_defaults()
    sns.set_style(style = "darkgrid", rc = None)

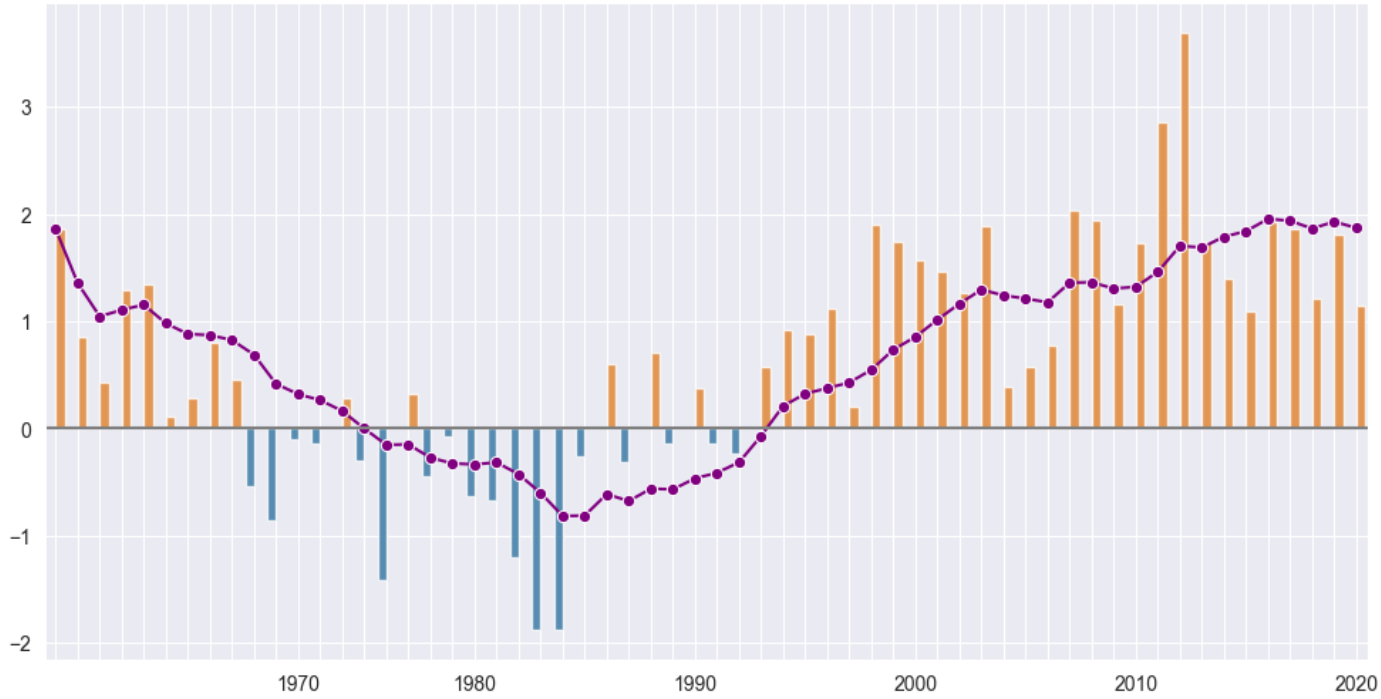
    fig, ax1 = plt.subplots(figsize = (12, 6))

    sns.lineplot(data = rolling_avg, marker='o', sort = False, ax = ax1, color = "purple")

    proper_colors = [1 if d >= 0 else 0 for d in diffs]

    g = sns.barplot(x = valid_years, y = diffs, alpha = 0.8, ax = ax1, hue = proper_colors, palette = sns.color_palette())
    g.legend_.remove()
    for label in g.get_xticklabels():
        ## label is a Text object (from matplotlib I believe)
        if int(label.get_text()) % 10 == 0:  ## only every 10th label is kept
            label.set_visible(True)
        else:
            label.set_visible(False)
    ax1.grid(True)
    plt.axhline(0, color = 'gray')
```

Out[33]: <matplotlib.lines.Line2D at 0x1f389877400>



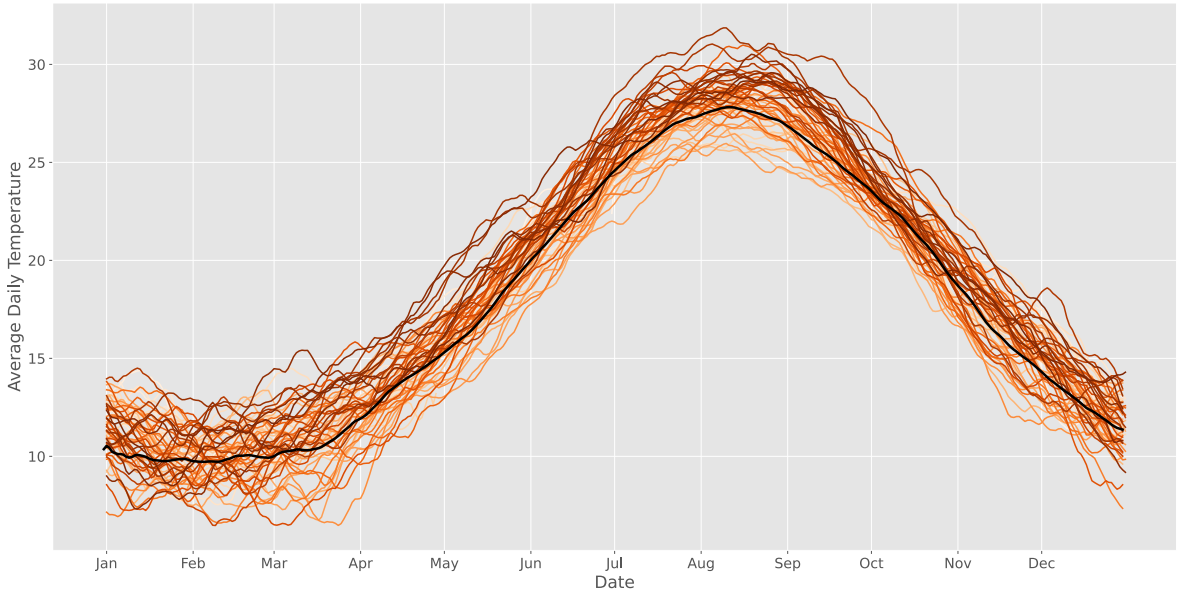
Ερμηνεία

Φαίνεται πως η απόκλιση από την μέση θερμοκρασία αναφοράς βαίνει αύξουσα με την πάροδο των ετών. Το θερμότερο καταγεγραμμένο καλοκαίρι κατά μέσο όρο ήταν το 2013. Αυτό βέβαια ίσως οφείλεται στην διαφορετική προέλευση των περισσότερων μετρήσεων για το έτος αυτό, όμως το γενικό συμπέρασμα παραμένει το ίδιο.

Ε3: Εξέλιξη Ημερήσιων Θερμοκρασιών

Θα βρείτε τη μέση θερμοκρασία κάθε ημέρας για κάθε έτος στην περίοδο από το 1955 έως το 2020. Στη συνέχεια θα δημιουργήσετε ένα γράφημα με τη μέση θερμοκρασία κάθε ημέρας για κάθε έτος. Το γράφημα θα έχει μία γραμμή για κάθε χρονιά, και θα έχει εξομαλυνθεί παίρνοντας ένα κυλιομένο παράθυρο 30 ημερών (30 days rolling average). Η κάθε γραμμή θα έχει διαφορετικό χρώμα, από το πιο ανοιχτό στο πιο σκούρο, καθώς αυξάνεται το έτος, όπως βλέπετε στο διάγραμμα που ακολουθεί.

Στο γράφημα θα προσθέσετε μια γραμμή που θα δείχνει τη μέση θερμοκρασία αναφοράς κάθε ημέρας για την περίοδο 1974-1999 (είναι η μαύρη γραμμή στο παρακάτω διάγραμμα). Η γραμμή αυτή θα έχει επίσης εξομαλυνθεί με ένα κυλιόμενο παράθυρο 30 ημερών. Πώς ερμηνεύετε την εικόνα;



Κατασκευάζω νέο dataframe με τα δεδομένα στην κατάλληλη αναπαράσταση - διάταξη ώστε να παραχθεί το απαιτούμενο διάγραμμα από το seaborn.

In [34]:
```
all_years = {}
missing_dates3 = []
null_temps3 = []
for year in range(1955, 2021):
    year_temps = {}
    start_date = date(year, 1, 1)
    end_date = date(year, 12, 31)

    delta = end_date - start_date ## returns timedelta object

    for i in range(delta.days + 1):
        day = start_date + timedelta(days = i)
        absolute_day = day - start_date
        day_string = day.isoformat()
        if(len(fts.loc[fts["DATE"] == day_string]) == 0): missing_dates3.append(day_string)
        elif(pd.isnull(fts.loc[fts["DATE"] == day_string]["TAVG"]).bool()): null_temps3.append(day_string)
        else:
            year_temps.update({absolute_day.days : fts.loc[fts["DATE"] == day_string]["TAVG"].values[0]})

    if(year_temps):
        all_years.update({year : year_temps})
```

In [35]:
```
newdict = {}
for year, temps in all_years.items():
    rolling_avg = {}
    tempvals = list(temps.values())
    for daydate, i in zip(temps.keys(), range(len(tempvals))):
        left = max(0, i - 29)
        rolling_avg.update({daydate: sum(tempvals[left : i + 1]) / (i - left + 1)})
    newdict.update({year : rolling_avg})
```

In [36]:
```
df = pd.DataFrame(columns = ["year", "date", "temp"])
for year, temps in newdict.items():
    for daydate, temp in temps.items():
        df = df.append({"year": int(year), "date": int(daydate), "temp": temp}, ignore_index = True)
```

In [38]:
```
df.sort_values(["year", "date"], inplace = True, ascending = True)
```

Υπολογίζω κυλιόμενους αριθμητικούς μέσους, και κατασκευάζω αντιστοίχιση αύξοντος αριθμού ημέρας με μήνα, για τις ενδείξεις στο διάγραμμα.

In [39]:
```
day_temps = {}
day_measurements = {}
for year in range(1974, 2000):
    start_date = date(year, 1, 1)
    end_date = date(year, 12, 31)

    delta = end_date - start_date ## returns timedelta object

    for i in range(delta.days + 1):
        day = start_date + timedelta(days = i)
        absolute_day = day - start_date
        day_string = day.isoformat()
        if(len(fts.loc[fts["DATE"] == day_string]) == 0): continue
        elif(pd.isnull(fts.loc[fts["DATE"] == day_string]["TAVG"]).bool()): continue
        else:
            if(absolute_day.days in day_temps.keys()):
                day_temps[absolute_day.days] += fts.loc[fts["DATE"] == day_string]["TAVG"].values[0]
                day_measurements[absolute_day.days] += 1
            else:
                day_temps.update({absolute_day.days : fts.loc[fts["DATE"] == day_string]["TAVG"].values[0]})
                day_measurements.update({absolute_day.days : 1})
```

In [40]:
```
averages = {}
for ad in day_temps.keys():
    averages.update({ad : day_temps[ad] / day_measurements[ad]})
```

In [41]:
```
rolling_avg = {}
for i, ad in enumerate(sorted(list(averages.keys()))):
    left = max(0, i - 29)
    rolling_avg.update({ad: sum([averages[j] for j in range(left, i + 1)]) / (i - left + 1)})
```

In [42]:
```
## still using 3.9 so I don't have match
def foo(x):
    if(x == 0): return "Jan"
    elif(x == 30): return "Feb"
    elif(x == 58): return "Mar"
    elif(x == 89): return "Apr"
    elif(x == 119): return "May"
    elif(x == 150): return "Jun"
    elif(x == 180): return "Jul"
    elif(x == 211): return "Aug"
    elif(x == 242): return "Sep"
    elif(x == 272): return "Oct"
    elif(x == 303): return "Nov"
    elif(x == 333): return "Dec"
    return None
```

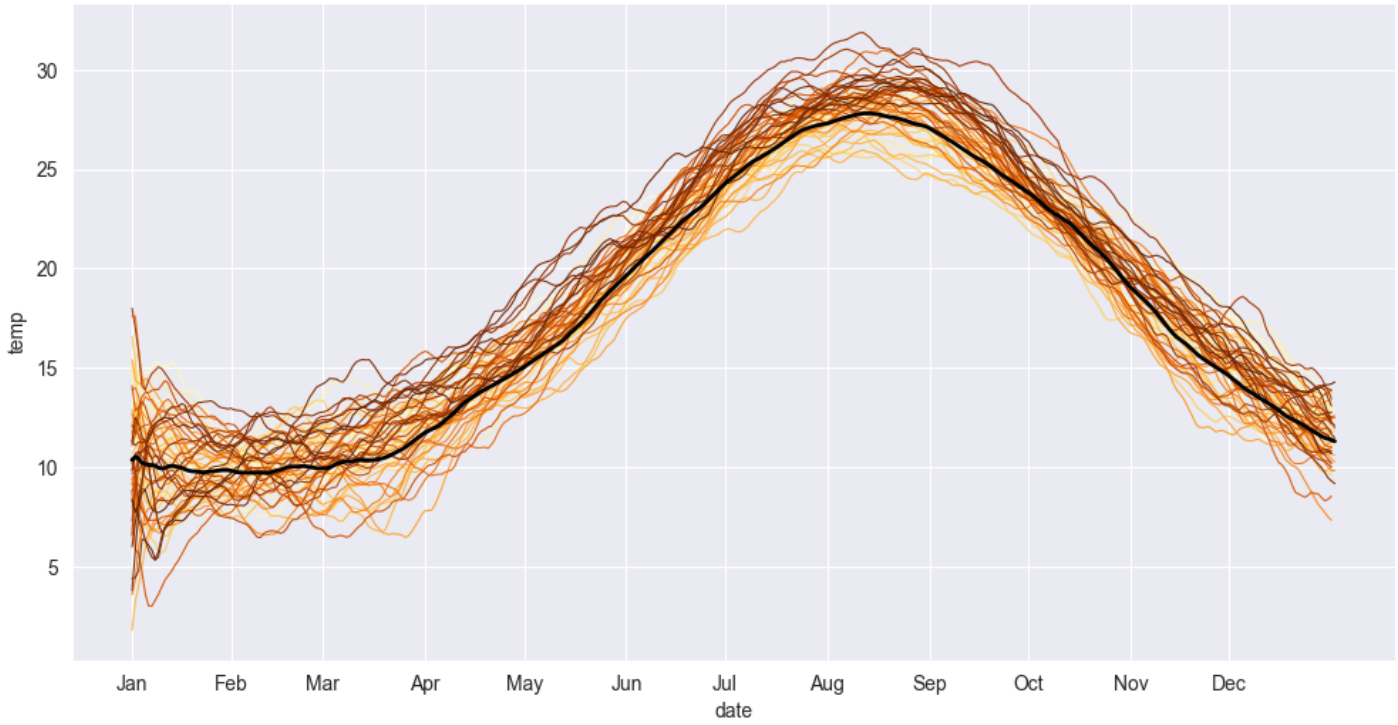
Διάγραμμα

In [43]:
```
## https://seaborn.pydata.org/examples/timeseries_facets.html
## https://seaborn.pydata.org/generated/seaborn.lineplot.html
fig, ax1 = plt.subplots(figsize = (12, 6))

g = sns.lineplot(data = df, x = "date", y = "temp", units = "year", estimator = None, linewidth = 0.75, hue = "year", palette = s
tickvals = [0, 30, 58, 89, 119, 150, 180, 211, 242, 272, 303, 333]
g.set_xticks(ticks = tickvals)
g.set_xticklabels(labels = list(map(foo, tickvals)))

sns.lineplot(data = rolling_avg, ax = ax1, color = "black", linewidth = 2)

ax1.grid(True)
g.legend_.remove()
```



Ερμηνεία

Το γενικό σχήμα του όλων των καμπυλών περιγράφει την μεταβολή της μέσης θερμοκρασίας στην διάρκεια του έτους. Άξιο παρατήρησης ότι οι σκουρόχρωμες καμπύλες, που αντιστοιχούν σε μεγαλύτερα έτη, βρίσκονται ψηλότερα στον κατακόρυφο άξονα. Συνεπώς με την πάροδο του χρόνου παρατηρείται κατακόρυφη μετατόπιση προς τα άνω, αύξηση δηλαδή, της μέσης θερμοκρασίας, καθ' όλη την διάρκεια του έτους.

Οδηγίες Υποβολής

Θα πρέπει να υποβάλετε την εργασία σας με τη μορφή σημειωματαρίου (Jupyter notebook), το οποίο θα περιέχει το σύνολο του κώδικα και την ανάλυσή σας. Αυτό σημαίνει ότι το σημειωματάριο θα πρέπει να διαβάζεται και να γίνεται κατανοητό και ως κείμενο (αλλιώς η υποβολή θα ήταν απλώς ένα αρχείο Python). Ο σκοπός δεν είναι μόνο να δείξετε ότι ξέρετε να γράφετε κώδικα, αλλά να εξηγήσετε και να πείσετε τον αναγνώστη. Θα πρέπει να υποβάλετε και όποια αρχεία δεδομένων χρησιμοποιείτε και να εξασφαλίσετε ότι το σημειωματάριο μπορεί να τρέξει χωρίς καμμία μετατροπή σε έναν άλλο υπολογιστή εκτός του δικού σας. Προσέξτε ιδιαιτέρως τα μονοπάτια ανάγνωσης των αρχείων, αυτά πρέπει να είναι σχετικά (relative paths) και όχι απόλυτα (absolute paths).

Κώδικας Τιμής

Συνειδητοποιείτε ότι αυτή είναι ατομική εργασία και θα πρέπει να εργαστείτε μόνοι σας. Μπορείτε βεβαίως να αναζητήσετε βοήθεια στο διαδίκτυο, αλλά αυτή θα πρέπει να αφορά τη χρήση των εργαλείων, και ιδιωμάτων, δεν θα πρέπει να αναζητείτε την απάντηση στα ερωτήματα της εργασίας. Μπορείτε να συζητήσετε με άλλους που παρακολουθούν το μάθημα προκειμένου να λύσετε μεταξύ σας απορίες κατανόησης, αλλά όχι μεθόδους επίλυσης. Αναλογιστείτε ότι σκοπός του μαθήματος είναι να αποκομίσετε κάποιες γνώσεις, και όχι να ξοδέψετε απλώς τον χρόνο και τα χρήματά σας.